AI AGENT · AUTONOMOUS OPERATIONS & CONTROL TOWER

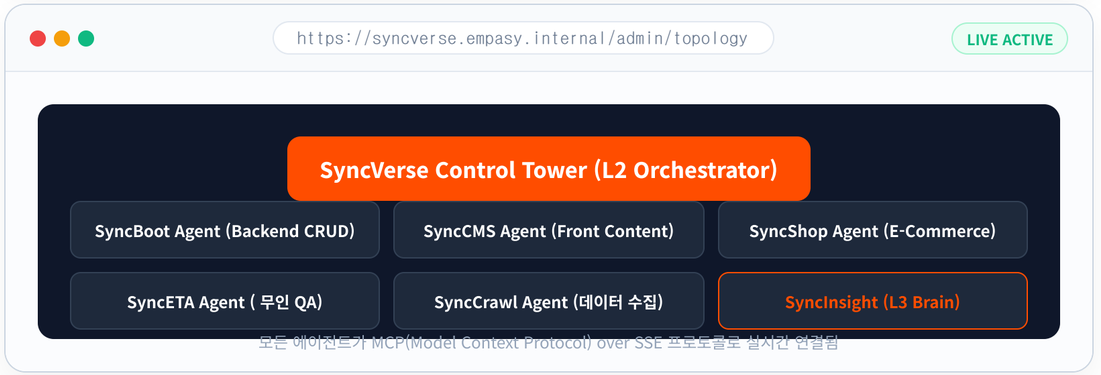
SyncVerse

AI Agent 자율 운영 오케스트레이션 및 중앙 관제탑 (Central Control Tower & AI DLC)

Multi-Agent Swarm Orchestrator · A2A MCP Protocol · Intent Routing · SDK Coding Agent · Self-Healing Pipeline

Intent Router Agent · Swarm Orchestrator Agent · Coding Dispatcher Agent · Self-Healing Supervisor Agent · FinOps Gateway Agent





엠파시 솔루션개발팀
Empasy.

자연어 요구사항부터 소스 수정, 자가치유 배포까지 총괄하는 관제탑입니다.

개별 AI 에이전트들을 하나로 묶어 소프트웨어 생애주기 전반(AI DLC)을 무인화합니다. 현업의 자연어 지시를 해석해 개발과 운영을 지휘하는 핵심 두뇌입니다.

한 문장으로 요약

A2A(MCP) 프로토콜로 모든 하위 에이전트를 조율하고, 의도 분류·SDK 자율 코딩·자가치유 파이프라인을 총괄하는 차세대 AI 자율 운영 플랫폼

- **Zero-Developer 파이프라인 :** 자연어 요구사항을 코딩 에이전트(SDK)로 자율 구현
- **자가치유(Self-Healing) 보장 :** SyncETA QA 오류 감지 시 서버 로그 수집 후 코드 자동 수정
- **엔터프라이즈 HITL 거버넌스 :** 핵심 DDL 변경 및 상용 배포는 인간 아키텍트 승인 필수

01 자연어 의도 분류 및 태스크 라우팅

지시가 '운영(CRUD)'인지 '개발(소스수정)'인지 실시간 판별하여 최적의 에이전트에 위임합니다.

02 Multi-Agent Swarm 오케스트레이션

표준 MCP 프로토콜을 통해 하위 에이전트(SyncBoot, CMS, Shop, ETA) 간 작업 체인을 조율합니다.

03 Google Antigravity SDK 코딩 디스패치

소스코드 수정이 필요한 경우 전문 코딩 에이전트를 호출하여 작업 브랜치에 코드를 자율 생성합니다.

04 무인 CI/CD 및 자가치유 복구 루프

빌드·테스트 오류 발생 시 클러스터 로그를 수집하여 코드를 스스로 고치고 배포합니다.

에이전트 군단을 지휘하여, 요구사항을 코드와 배포로 즉시 변환합니다.

SyncVerse의 워크플로우는 세 가지 단계로 나뉩니다. 자연어 지시를 분석하여 의도를 분류하고(A), 다중 에이전트 군단과 SDK 코딩을 조율하며(B), 무인 검증 배포 및 분산 사가 트랜잭션으로 시스템을 완성합니다(C).

PRE-PROCESS

A. 의도 분류 및 요구사항 정밀 분석

요구사항 접수 시 자연어를 분석하여 운영(CRUD)과 개발(Code) 경로를 분기합니다.

자연어 인텐트 라우팅 (Intent Routing)
- 운영 데이터 수정 vs 소스코드 변경 자동 판별

작업 분할 및 에이전트 매핑
- 요구사항을 하위 에이전트별 단위 태스크로 분해

컨텍스트 및 보안 정책 주입
- 도메인 규칙 및 테넌트 권한 컨텍스트 사전 검증

CORE EXECUTION

B. Swarm 조율 & 자율 코딩/자가치유

MCP 프로토콜로 에이전트를 가동하고 SDK 코딩 에이전트로 소스를 수정합니다.

1. 다섯 에이전트 군단 조율
- 라우터-오케스트레이터-코딩-치유-FinOps 유기 결합

2. SyncSdk 코딩 에이전트 가동
- Antigravity SDK / Open-SWE를 호출하여 코드 수정

3. 자가치유(Self-Healing) 루프
- SyncETA 오류 시 클러스터 로그 수집 후 코드 재수정

POST-PROCESS

C. 검증 배포 및 Saga 트랜잭션

무인 CI/CD로 프리뷰 배포 후 인간 검증 및 분산 트랜잭션을 확정합니다.

무인 CI/CD 파이프라인 연동
- Git PR 생성, 자동 빌드 및 Stage 환경 배포

인간 아키텍트 승인 (HITL)
- DDL 변경 및 상용 배포 전 최종 1-Click 승인

Saga 분산 트랜잭션 제어
- 다중 시스템 간 데이터 정합성 보장 및 롤백

에이전트 다섯이 중앙 관제탑을 이루어 자율 운영합니다.

사용자 요구 한 건이 들어오면 각 관제 에이전트가 순서대로 판단하고 하위 워커를 지휘합니다.

01 **Intent Router Agent (의도 분류 및 라우팅)**
자연어 요구사항이 '운영(CRUD)'인지 '개발(소스수정)'인지 1초 내 판별하여 경로를 지정합니다.
AI 사용 · 자동 수행

02 **Swarm Orchestrator Agent (다중 에이전트 군단 조율)**
MCP 프로토콜을 통해 SyncBoot, CMS, Shop 등 하위 에이전트 간 순차/병렬 작업을 조율합니다.
AI 사용 · 자동 수행

03 **Coding Dispatcher Agent (SyncSdk 코딩 에이전트 디스패치)**
소스코드 수정 시 Google Antigravity SDK를 호출하여 작업 브랜치에 코드를 자율 수정합니다.
AI 사용 · 확인 1회

04 **Self-Healing Supervisor (자가치유 및 로그 분석)**
SyncETA로부터 QA 실패 보고를 받으면 클러스터 로그를 수집해 코딩 에이전트에 원인을 전달합니다.
AI 사용 · 자동 수행

05 **FinOps Gateway Agent (SyncLLM 게이트웨이 & 비용 통제)**
다양한 LLM API를 단일 게이트웨이로 통합하고 시맨틱 캐싱과 토큰 비용을 실시간 최적화합니다.
AI 사용 · 자동 수행

[확장성] 새로운 도메인 에이전트(예: 물류, 금융, CRM)가 추가되어도 표준 MCP 프로토콜만 준수하면 SyncVerse 관제탑에 플러그앤플레이로 즉시 편입됩니다.

에이전트와 시스템이 제각각 놀아 중앙 통제가 불가능합니다.

여러 AI 도구와 사내 시스템이 파편화되어 있으면 맥락 공유가 안 되고, 버그 발생 시 사람이 일일이 로그를 뒤져 코드를 고쳐야 합니다.

01 **단순 기능 수정 요청에도 기획-개발-QA 간 긴 커뮤니케이션 지연**
간단한 화면 수정 하나에도 지라 티켓 발행, 개발자 배정, 회의로 수일의 시간 낭비

02 **파편화된 에이전트로 인한 중앙 거버넌스 및 맥락(Context) 부재**
개별 툴마다 맥락이 단절되어 전체 비즈니스 플로우 관점의 자율 운영 불가능

03 **버그 발생 시 로그 수집, 원인 분석, 코드 수정, 재배포의 긴 리드타임**
QA 실패 시 개발자가 로그를 모아 디버깅하고 코드를 다시 짜 배포하느라 수일 소요

04 **LLM API 호출 비용 급증 및 벤더 종속성(Lock-in) 심화**
여러 팀에서 각자 LLM을 호출하여 중복 비용 발생 및 프롬프트 캐싱 부재

05 **마이크로서비스 간 분산 트랜잭션 실패 시 데이터 불일치 위험**
여러 시스템에 걸친 작업 중 하나가 실패했을 때 롤백 처리가 어려워 데이터 오염 발생

AI는 소스 수정과 자가치유를 맡습니다. 아키텍처 승인은 사람이 합니다.

AI에게 '신규 기능을 추가하고 테스트 실패 시 코드를 고쳐라'라고 하면 잘합니다. 상용 배포와 핵심 DDL 반영은 시스템 아키텍트가 승인합니다.

01 INTENT / SWARM

의도 분류 및 다중 에이전트 작업 조율

- 자연어 요구사항 정밀 분석 및 태스크 분할
- MCP 기반 하위 에이전트 체인 실행
- SyncLLM 기반 비용 최적 모델 라우팅

02 HUMAN IN THE LOOP

시스템 아키텍트 (사람)

- AI가 제안한 DDL 및 아키텍처 변경안 검토
- Dev 검증 완료 후 상용 배포 최종 승인
- 가장 핵심적인 안전 거버넌스 지점

03 CODING / SELF-HEALING

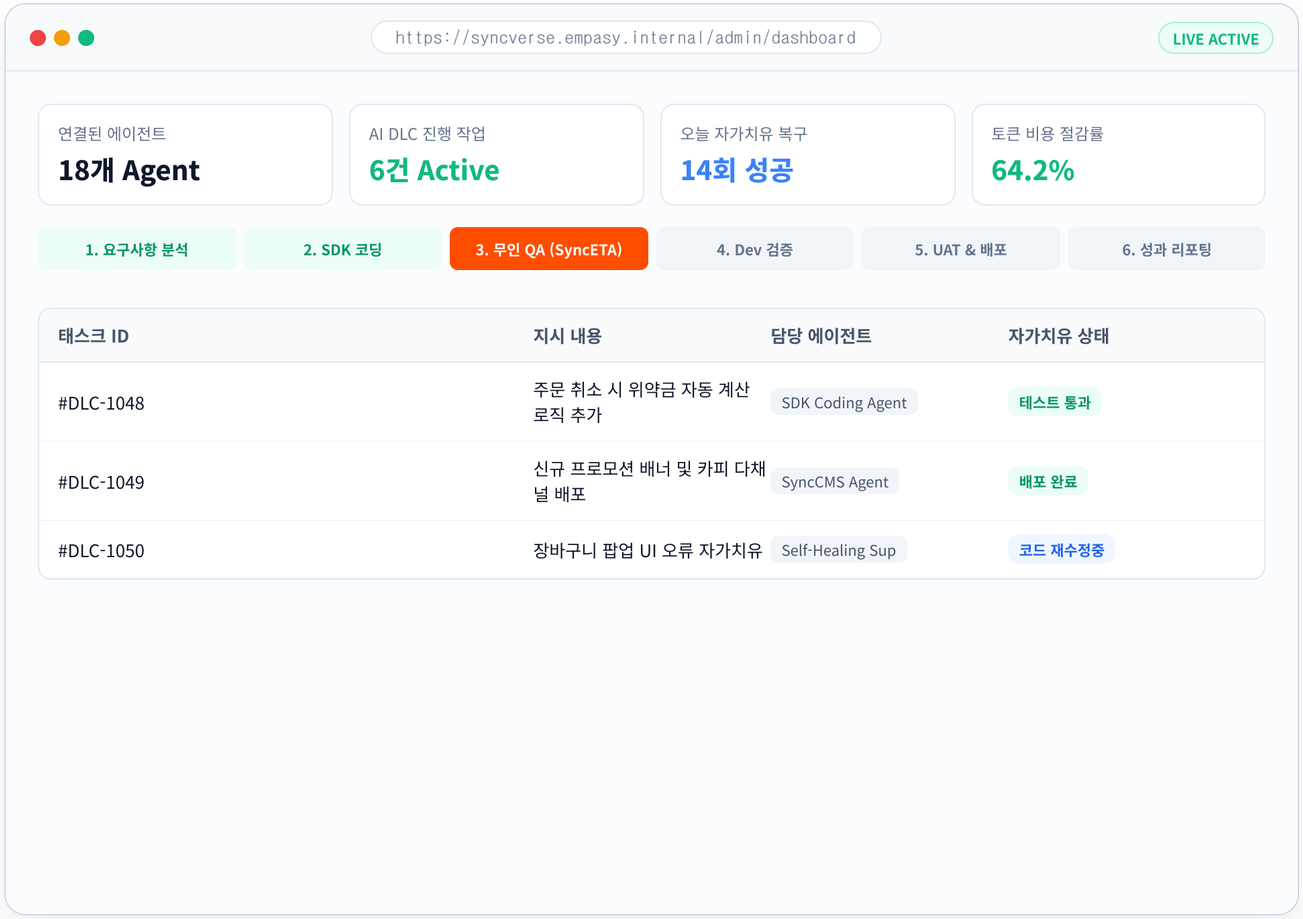
자율 코딩 및 무인 자가치유 배포

- Antigravity SDK 기반 소스코드 자율 수정
- SyncETA QA 오류 시 클러스터 로그 기반 자가치유
- Saga 분산 트랜잭션 및 원클릭 롤백

SyncVerse를 쓰면: 현업 담당자는 자연어로 요구하고, 개발자는 AI가 작성한 코드의 안전성만 확인하면, 소스 수정부터 빌드, QA 테스트, 자가치유 재배포까지 전 과정이 무인 완결됩니다.

전사 AI 에이전트 군단과 소프트웨어 생애주기를 실시간 관제합니다.

요구사항 분석부터 코딩, QA 검증, 상용 배포까지 6-Step AI DLC 파이프라인의 진행 상황을 한눈에 파악합니다.



이 화면에서 할 수 있는 일

01 글로벌 Swarm 활성 상태 관제
연결된 모든 Layer 1 워커 에이전트(SyncBoot, CMS 등)의 실시간 상태 모니터링

02 6-Step AI DLC 파이프라인 추적
요구사항 -> 코딩 -> 무인QA -> Dev검증 -> UAT -> 결과리포트 진행률 표시

03 실시간 자가치유(Self-Healing) 현황
QA 오류 발생 및 SDK 자동 수정 복구 진행 상태 실시간 감지

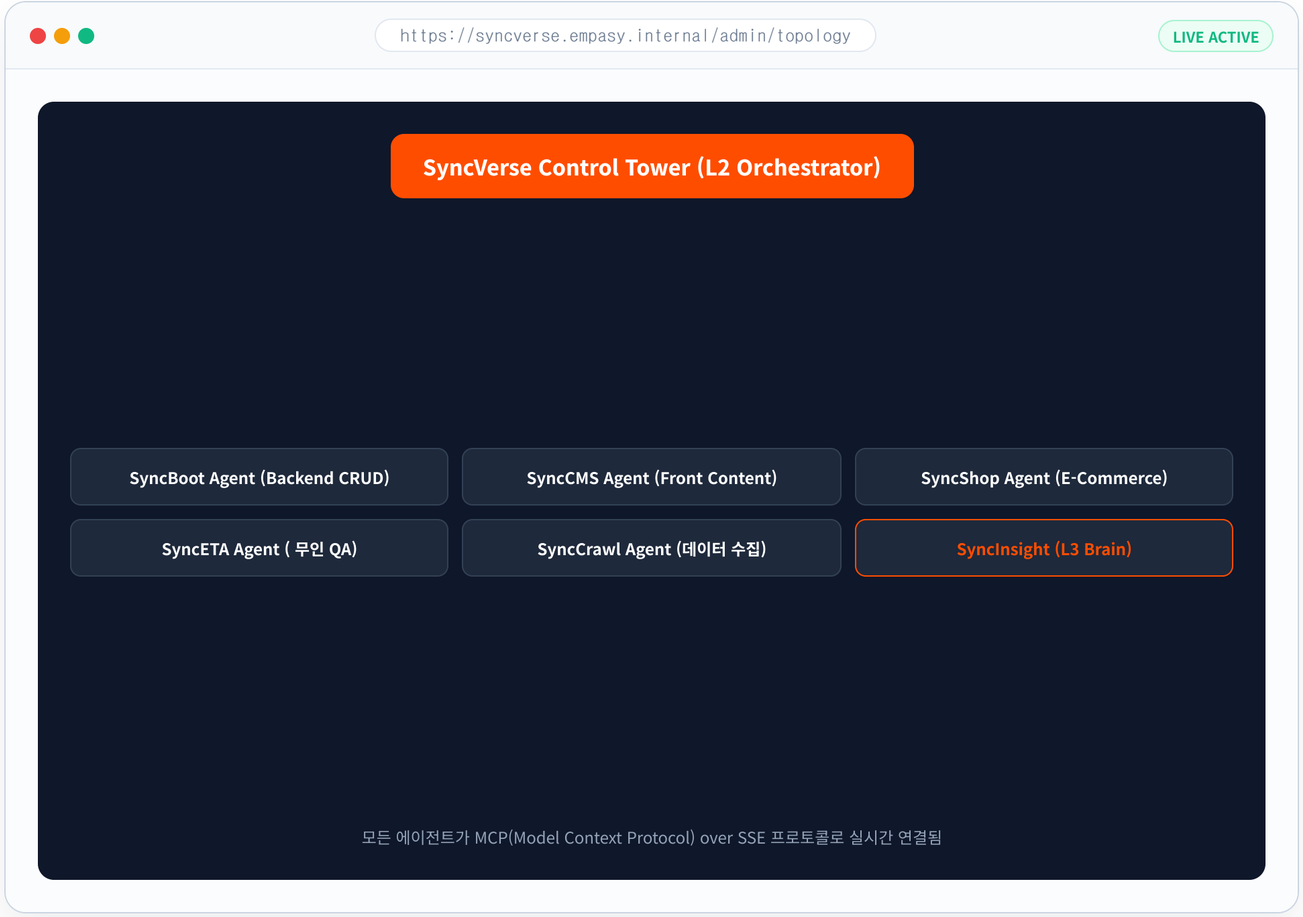
04 LLM 토큰 소비 및 비용 게이지
금일 전체 에이전트의 토큰 사용량 및 예산 대비 소진율 시각화

05 미승인 HITL 대기 알림
아키텍트의 승인이 필요한 DDL 및 배포 요청 목록 즉시 노출

06 비상 전체 정지(Kill-Switch)
이상 동작 감지 시 모든 에이전트의 작업을 즉시 안전하게 중단

에이전트 간의 A2A 협업 관계와 데이터 흐름을 시각적으로 설계합니다.

Vue Flow 기반의 노드-엣지 인터페이스로 에이전트 간 MCP 호출 경로와 의존성을 인터랙티브하게 조율합니다.



이 화면에서 할 수 있는 일

01 노드-엣지 기반 A2A 플로우 뷰
에이전트 간 MCP 메시지 및 데이터 전달 경로를 그래프로 한눈에 파악

02 동적 에이전트 추가 및 연결
신규 MCP 에이전트를 캔버스에 드래그하여 기존 워크플로우에 편입

03 실시간 패킷 & 메시지 스트림
에이전트 간 주고받는 JSON-RPC 요청/응답 페이로드를 실시간 확인

04 조건부 분기 및 병렬 실행 제어
특정 조건(성공/실패)에 따른 에이전트 실행 분기(If-Else) 시각적 설정

05 지연시간(Latency) 병목 분석
각 에이전트 노드별 응답 시간을 히트맵으로 표시하여 병목 구간 식별

06 토폴로지 템플릿 저장 및 공유
검증된 에이전트 협업 구성을 템플릿으로 저장하여 재사용

자연어 요구사항을 분석하여 운영과 개발 작업을 완벽히 분배합니다.

입력된 프롬프트의 의도를 분석하여 소스코드 수정이 필요한지, 단순 데이터 조작인지를 자동 라우팅합니다.

https://syncverse.empasy.internal/admin/intent

LIVE ACTIVE

사용자 자연어 지시:

"VIP 고객 대상 정기 구독 결제 시 10% 추가 포인트 적립 기능을 추가하고 테스트해줘."

의도 분류: **[개발 / 소스코드 변경 (CODE_CHANGE)]** 신뢰도: 99.4%
할당 파이프라인:
1. Google Antigravity SDK 코딩 에이전트 호출
2. SyncETA 결제 시나리오 자동 검증

이 화면에서 할 수 있는 일

01 자연어 의도(Intent) 자동 분류
요청 텍스트를 분석하여 CRUD, Code Fix, New Feature, QA 등으로 분류

02 의도 분류 신뢰도(Confidence) 점수
AI의 분류 확신 점수를 표시하고 임계값 미만 시 사용자에게 재확인

03 라우팅 규칙 커스텀 튜닝
기업 고유의 키워드 및 업무 규칙에 맞춘 라우팅 우선순위 설정

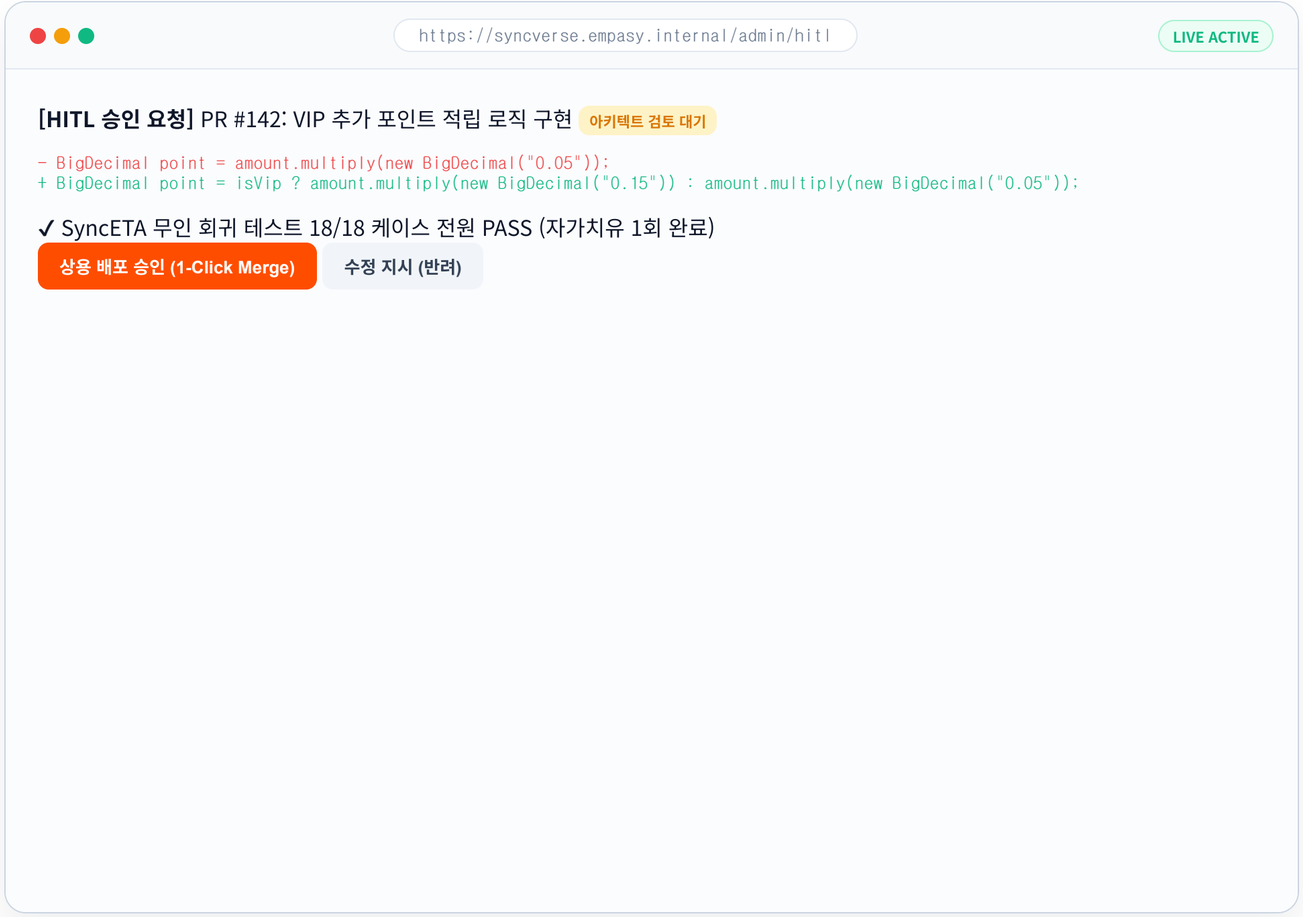
04 다중 태스크 자동 분해
복합 요청('상품 등록하고 결제 로직 수정해줘')을 개별 단위 작업으로 분할

05 컨텍스트 RAG 프롬프트 보강
사내 프로젝트 아키텍처 문서를 프롬프트에 자동으로 주입하여 지시

06 라우팅 히스토리 및 통계
의도 분류 성공률 및 에이전트별 작업 배분 비율 통계 제공

AI의 DDL 제안과 코드 수정을 검토하고 원클릭으로 승인합니다.

Antigravity SDK가 생성한 코드 Diff와 SyncETA의 자가치유 피드백 로그를 한 화면에서 확인하고 승인합니다.



이 화면에서 할 수 있는 일

01 코드 Diff & DDL 변경안 뷰어
AI 코딩 에이전트가 수정한 소스코드와 DDL을 나란히 대조 검토

02 원클릭 승인(Approve) 및 반려
버튼 클릭 한 번으로 Git PR 병합 및 자동 배포 파이프라인 트리거

03 자가치유(Self-Healing) 상세 타임라인
SyncETA 오류 감지 -> 로그 수집 -> SDK 재수정 -> 재검증 과정 추적

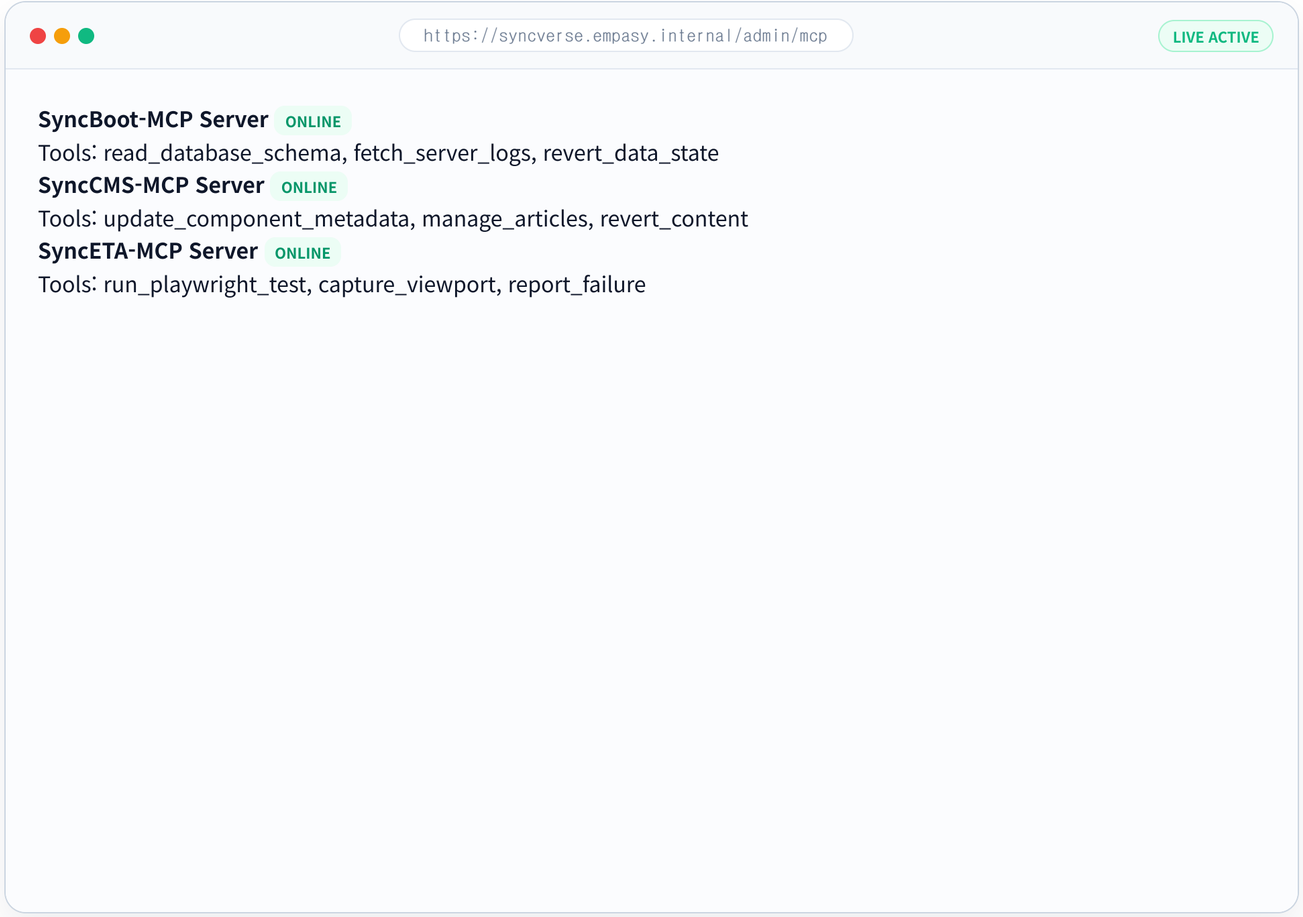
04 클러스터 분산 로그 뷰어
자가치유에 사용된 개별 서버 인스턴스의 로그 스니펫 즉시 확인

05 개발자 코멘트 및 수정 지시
반려 시 AI 코딩 에이전트에게 추가 피드백 프롬프트를 전달하여 재작업

06 상용(Prod) 배포 2단계 인증
중요 프로덕션 배포 시 OTP 및 다중 결재자 승인 체계 지원

전사 에이전트가 사용하는 표준 MCP 도구를 통합 카탈로그로 관리합니다.

Model Context Protocol 기반의 Tool, Resource, Prompt 템플릿을 등록하고 호출 권한을 통제합니다.



이 화면에서 할 수 있는 일

01 전사 MCP 서버 및 도구 카탈로그
등록된 모든 MCP Server와 가용 Tool 목록을 실시간 탐색

02 JSON Schema 기반 입출력 검증
각 도구의 파라미터 스키마와 반환 데이터 포맷 자동 유효성 검사

03 인터랙티브 도구 테스트 샌드박스
웹 콘솔에서 도구를 직접 호출하고 JSON 결과를 테스트

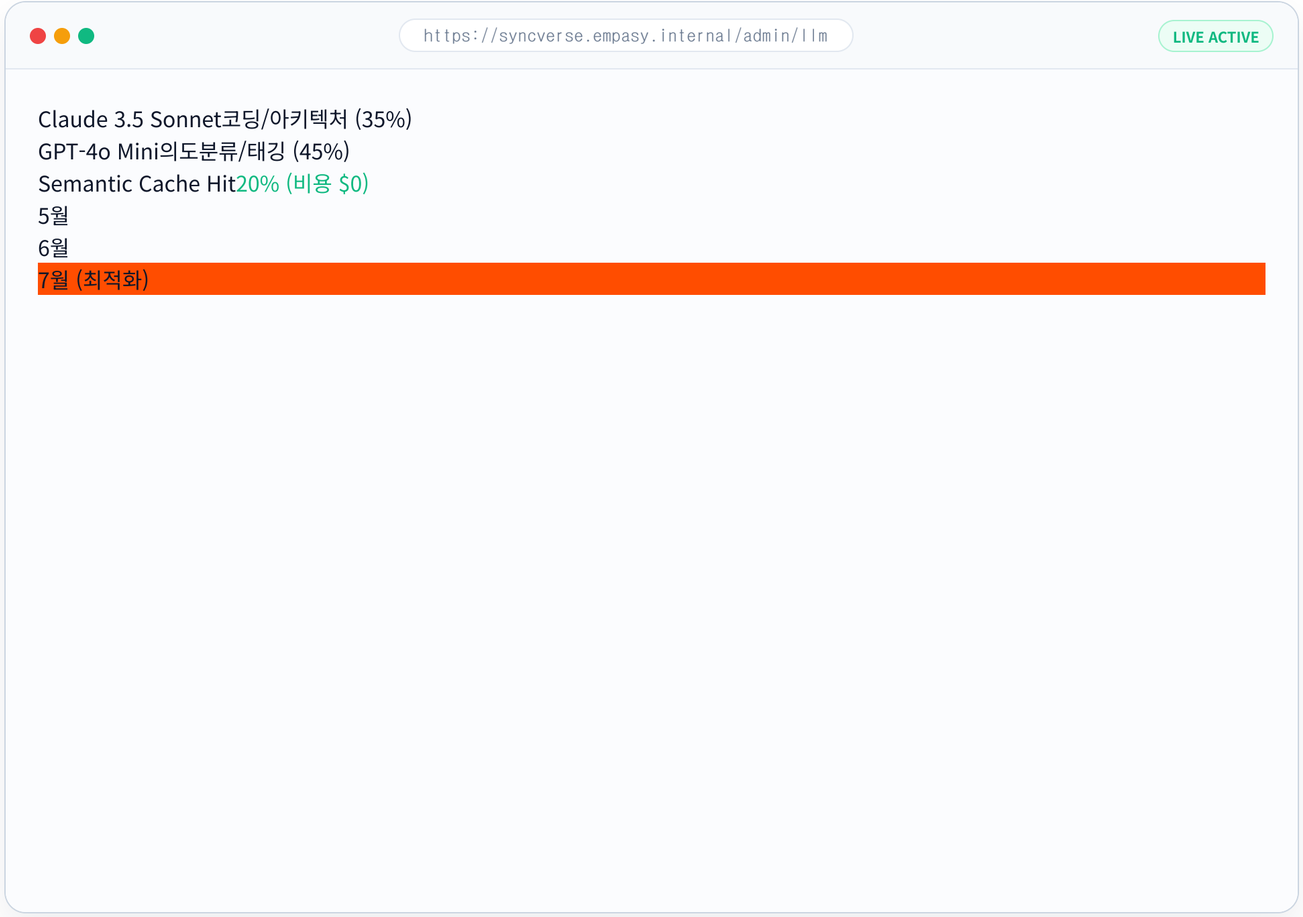
04 도구별 세분화 실행 권한(RBAC)
특정 에이전트나 사용자만 위험 도구(DDL 실행 등)를 호출하도록 제한

05 도구 호출 빈도 및 에러율 모니터링
각 MCP Tool의 분당 호출 수(RPM), 평균 응답시간, 실패율 집계

06 Swagger / OpenAPI 원클릭 MCP 변환
기존 사내 REST API 문서를 표준 MCP 도구로 자동 변환 등록

다중 LLM 호출을 최적화하여 AI 토큰 비용을 60% 이상 절감합니다.

Claude, GPT, 오픈소스 LLM을 단일 엔드포인트로 통합하고 시맨틱 캐싱과 예산 통제를 수행합니다.



이 화면에서 할 수 있는 일

01 지능형 멀티 모델 라우팅
작업 난이도에 따라 고성능 모델과 경량 모델로 자동 분기 호출

02 시맨틱 임베딩 캐싱 (Cache Lookup)
유사한 프롬프트 요청에 대해 기존 캐시 결과를 즉시 반환하여 비용 절감

03 실시간 PII 개인정보 마스킹
LLM 외부 전송 전 주민번호, API Key 등 민감정보 자동 마스킹

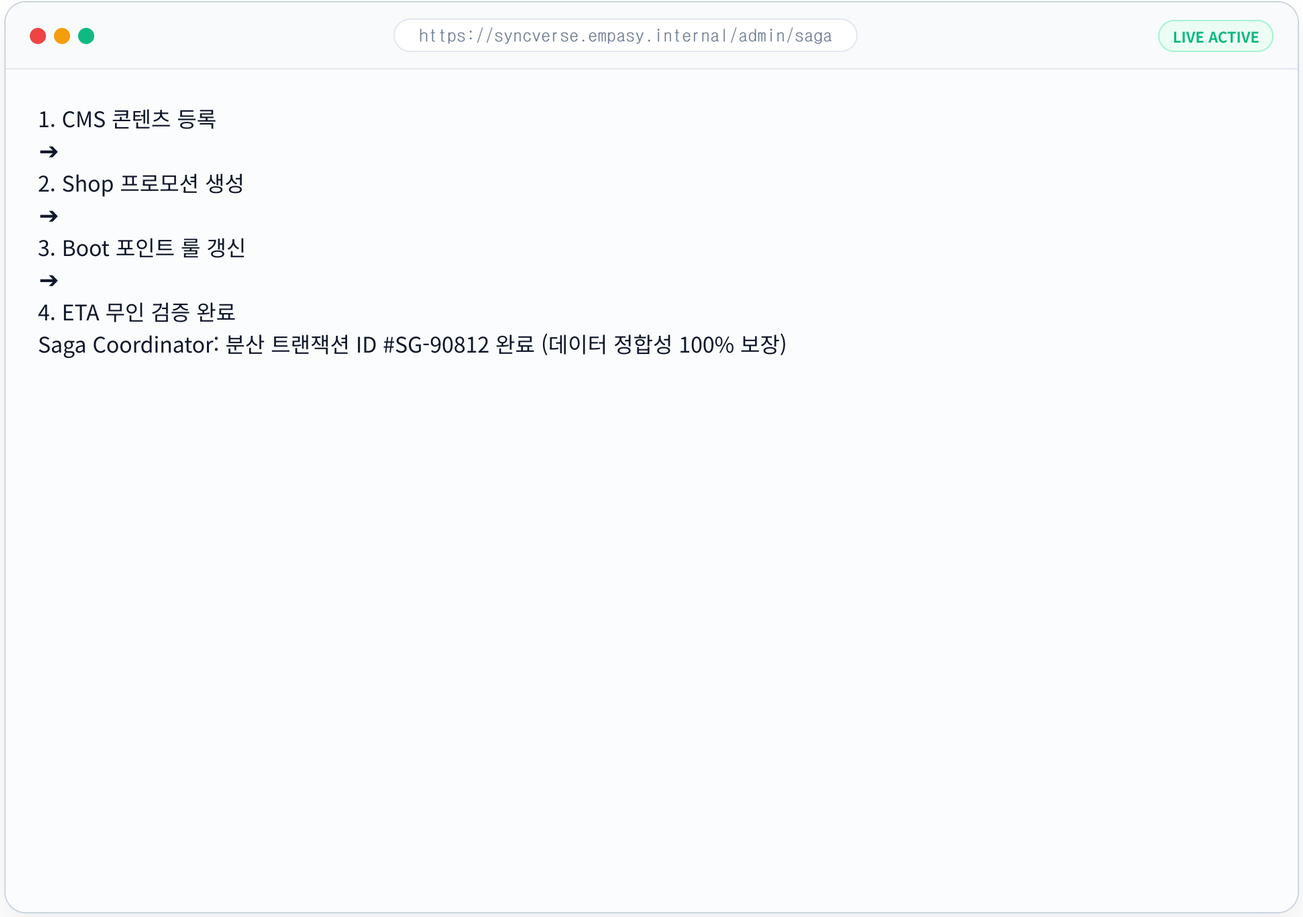
04 프로젝트별 토큰 예산(Budget) 한도
부서/프로젝트별 월간 토큰 한도를 설정하고 초과 시 자동 알림 및 차단

05 LLM 벤더 페일오버(Failover)
특정 모델 API 장애 시 대체 LLM(Anthropic <-> OpenAI)으로 자동 전환

06 비용 및 응답속도 분석 대시보드
모델별, 에이전트별 토큰 소비량 및 평균 지연시간 정밀 분석

다중 시스템에 걸친 작업의 정합성을 보장하고 장애 시 완벽히 롤백합니다.

분산 사가(Saga) 오케스트레이션을 통해 마이크로서비스 간 트랜잭션을 원자적으로 관리하고 감사 로그를 남깁니다.



이 화면에서 할 수 있는 일

01 분산 사가 트랜잭션 타임라인
여러 에이전트에 걸친 연쇄 작업의 스텝별 실행 상태 시각화

02 보상 트랜잭션(Compensating) 자동 실행
중간 단계 실패 시 이전 단계들을 역순으로 안전하게 롤백

03 작업 상태(State) 영속화
Redis 및 RDB를 활용하여 서버 재부팅 시에도 작업 상태 완벽 복구

04 전사 A2A 통신 감사 로그
누가 지시하고 어떤 에이전트가 어떤 도구를 실행했는지 전수 기록

05 트랜잭션 지연 및 타임아웃 제어
단계별 타임아웃 정책 설정 및 무한 루프 방지 서킷 브레이커

06 감사 보고서 및 규제 준수 증적
금융 및 공공 컴플라이언스 기준에 맞춘 실행 감사 로그 출력

아키텍트가 하는 일은 AI가 완성한 코드와 배포를 '승인'하는 것입니다.

의도를 분류하고, 소스코드를 수정하며, QA를 돌리고 자가치유하는 일은 AI 에이전트가 합니다. 담당자는 제안된 아키텍처와 상용 배포가 맞는지만 봐 주시면 됩니다.

STEP 01

INTENT / SDK / ETA

자연어 분석, 소스 수정 및 무인 QA

자연어 요구사항을 해석해 SDK 코딩 에이전트가 소스를 수정하고, SyncETA가 무인 테스트 및 자가치유를 완결합니다.

STEP 02 (주요 역할)

시스템 아키텍트 (사람)

코드 변경과 상용 배포를 승인합니다

AI가 완성한 소스코드 Diff와 Dev 환경 검증 결과를 확인하고 상용 배포를 승인합니다. 요청 한 건에 한 번 클릭합니다.

STEP 03

SWARM / SAGA / FINOPS

상용 배포를 완료하고 전사 기록함

승인된 코드를 상용 환경에 무중단 배포하고, 분산 트랜잭션을 확정하며, 토큰 비용과 감사 이력을 기록합니다.

완벽한 자율 운영 체계 — 개발 병목 현상이 사라집니다

- 자연어 요구가 소스코드 수정과 자가치유로 이어져 단순 유지보수 리드타임이 분 단위로 단축되기 때문입니다.

투명한 중앙 거버넌스 — 모든 시스템의 변경 이력이 남습니다

- 에이전트 간의 모든 통신과 코드 변경, 승인 이력이 중앙 관제탑에 일원화되어 보존됩니다.

지금 하시는 일과 비교하면 이렇습니다.

개발자가 사라지는 것이 아닙니다. 개발자는 단순 코더에서 AI를 지휘하는 '시스템 아키텍트'로 진화합니다.

지 금 (기존 수작업 개발/운영 방식)	SYNCVERSE 도입 후
요구사항 반영 주기 기획서 작성 및 회의를 거쳐 배포까지 수주 소요	**요구사항 반영 주기** 자연어 지시 후 SDK 코딩 및 무인 QA로 수분 내 반영
소스코드 수정 개발자가 직접 로컬 IDE를 켜고 코드를 수작업 수정	**소스코드 수정** Google Antigravity SDK 코딩 에이전트가 작업 브랜치 자율 수정
QA 및 버그 수정 QA 에러 발생 시 개발자가 로그 수집 및 수동 재디버깅	**QA 및 버그 수정** SyncETA 에러 보고 시 클러스터 로그 기반 자가치유 자동 수정
LLM 비용 관리 팀마다 개별 API 키를 사용해 중복 비용 및 통제 불가	**LLM 비용 관리** SyncLLM 단일 게이트웨이 및 시맨틱 캐싱으로 비용 60% 절감
에이전트 거버넌스 개별 AI 도구들이 파편화되어 맥락 공유 및 통제 불가	**에이전트 거버넌스** 표준 MCP 기반 중앙 관제탑으로 모든 에이전트 일원화 통제
배포 및 롤백 배포 실패 시 개발자가 야간 비상대기하며 수동 롤백	**배포 및 롤백** 무인 CI/CD 및 Saga 분산 트랜잭션으로 원클릭 안전 롤백

효과는 직접 써 보고 확인하시면 됩니다.

몇 퍼센트 줄어든다는 숫자를 먼저 말씀드리지 않겠습니다. 지금 분명히 말할 수 있는 것과, 직접 써 봐야 알 수 있는 것을 나눠 적었습니다.

지 금 확 실 한 것

- 기능 개발 및 배포 리드타임(TTM)이 획기적으로 압축됩니다.
- 자가치유 파이프라인으로 단순 버그 수정에 드는 개발 시간이 사라집니다.
- 중앙 집중식 HITL 승인 콘솔로 무단 코드 변경 사고를 원천 방지합니다.
- SyncLLM 게이트웨이를 통해 사내 AI 토큰 비용이 대폭 절감됩니다.
- 개발자는 단순 코딩에서 벗어나 핵심 비즈니스 아키텍처에 집중하게 됩니다.

써 보 고 재 야 아 는 것

- 자연어 요청 접수부터 상용 배포까지 걸리는 실제 사이클 타임
- SyncETA 테스트 실패 후 자가치유로 스스로 해결되는 버그의 비율
- 시맨틱 캐싱 및 모델 라우팅을 통해 절감된 실제 LLM API 비용
- 개발팀의 단순 유지보수 업무 감소에 따른 고부가가치 R&D 전환율

단순한 CI/CD 도구나 챗봇과는 차원이 다릅니다.

명령만 수행하는 도구가 아니라, 요구사항을 스스로 해석하고 코딩 에이전트를 가동하며 테스트 오류를 자가치유하는 AI DLC 플랫폼입니다.

01 A2A MCP 기반 재귀적 군단(Swarm)을 지휘합니다

L1(워커)-L2(관제탑)-L3(의사결정) 계층 구조로 에이전트가 다른 에이전트를 유기적으로 지휘합니다.

02 운영(CRUD)과 개발(Code)을 엄격히 분리하여 안전합니다

하위 에이전트는 소스를 만지지 못하게 하고, 코드 수정은 전용 SDK 코딩 에이전트만 수행합니다.

03 SyncETA 및 클러스터 로그 기반 자가치유를 실현합니다

테스트 실패 시 클러스터 서버 로그를 직접 수집하여 코드를 스스로 고치는 완전 무인 루프를 제공합니다.

04 엔터프라이즈 HITL 거버넌스로 안전을 보장합니다

아무리 뛰어난 AI라도 DDL 변경과 상용 배포는 인간 아키텍트의 명시적 승인을 거치도록 강제합니다.

05 SyncLLM을 통한 멀티모델 FinOps 최적화를 내장합니다

다양한 벤더의 LLM을 최적 비용으로 라우팅하고 시맨틱 캐싱으로 예산을 엄격히 통제합니다.

기존 웹/앱 시스템 한 개의 관제탑으로 시작합니다.

한 번에 전사 모든 인프라를 바꾸지 않습니다. 서비스 하나를 골라 에이전트들을 연결하고, 효과를 확인한 뒤 넓힙니다.

2 ~ 4주

첫 째

중앙 관제탑 셋업 & 에이전트 등록

- 대상 서비스 시스템 및 Git 저장소 연동
- SyncBoot, SyncCMS, SyncETA 에이전트 MCP 등록
- 인텐트 라우팅 규칙 및 프로젝트 컨텍스트 주입

4 ~ 8주

둘 째

자가치유 파이프라인 시범 운영

- 자연어 지시 기반 SDK 코딩 및 무인 QA 연동
- SyncETA 오류 감지 시 자가치유 코드 수정 검증
- 아키텍트 HITL 승인 워크플로우 및 TTM 단축 측정

이후

셋 째

전사 AI DLC 자율 운영 확대

- 전사 모든 서비스 및 마이크로서비스로 관제탑 확대
- SyncInsight 연동을 통한 비즈니스 의사결정-실행 루프 완성
- 전사 Zero-Developer Operations 생태계 달성

⚙ 이미 쓰고 계신 인프라와 연결 형상관리 (Git / GitHub / GitLab) · CI/CD (Jenkins / Azure DevOps / GitHub Actions) · 컨테이너 (Kubernetes / Docker) · 메신저 (Slack / Teams)

SyncVerse

감사합니다

SyncVerse — AI Agent 자율 운영 오케스트레이션 및 중앙 관제탑

엠파시 솔루션개발팀
Empasy.